

Instituto Tecnológico de Costa Rica
ESCUELA DE INGENIERÍA EN COMPUTADORES



**CE1107 Fundamentos de Arquitectura de
Computadores**

PROYECTO GRUPAL 1 - BITÁCORA

Máquinas de estados

Estudiante	Carnet
José Solano Mora	2018102753
Javier Alejandro Barraza Arias	2022114377
Amanda Jimena Quesada Porras	2023086337
Luis Adrian Prendas Arias	2023147608

Profesor:

Luis Alberto Chavarria Zamora

Primer Semestre, 2026

1 Diagramas de Diseño Estructural: Modelo Optimizado de 4 bits

1.1 Arquitectura Modular del Sistema (Módulo TOP_PWM)

Tras la validación de requerimientos, se definió una arquitectura modular jerárquica para el sistema de control de potencia. El diseño final se centra en la optimización de recursos mediante un bus de datos de 4 bits, lo cual es suficiente para representar los 16 niveles de cuantización del elevador y los 5 niveles de duty cycle exigidos para el proyecto.

El sistema se divide en tres dominios operativos principales:

1. **Dominio de Entrada:** Captura estática mediante switches (SW[3:0]) que simulan la distancia calculada por la ALU en el proyecto final.
2. **Dominio Temporal (pwm_mod):** Implementación secuencial de la base de tiempo y comparación de magnitud.
3. **Dominio de Visualización (hex_dec):** Decodificación combinacional para el monitoreo del estado en display de siete segmentos.

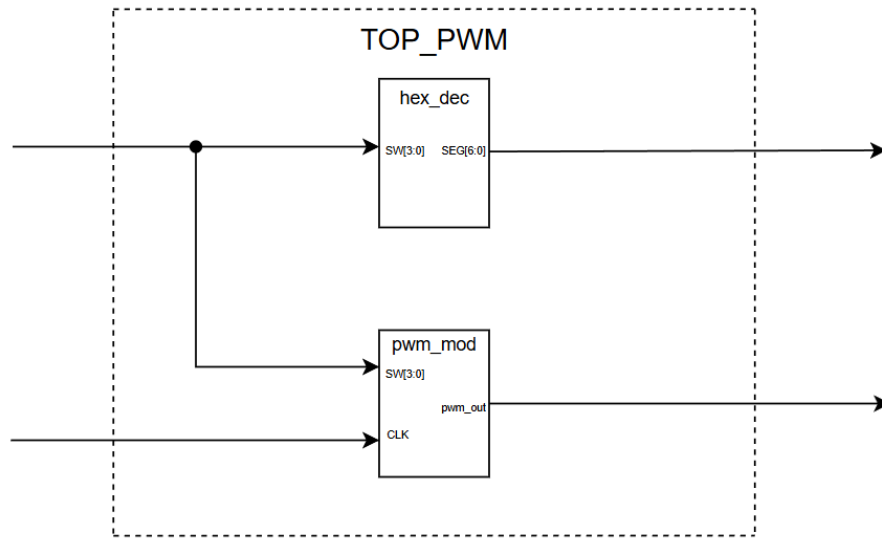


Figure 1: Diagrama de bloques general del módulo TOP_PWM

1.2 Detalle Interno del Módulo pwm_mod (4 bits)

El submódulo `pwm_mod` constituye el núcleo del diseño secuencial. A diferencia del prototipo inicial, este modelo utiliza un contador módulo 16, lo que permite una frecuencia de operación de 3.125 MHz (basado en el reloj base de 50 MHz), garantizando una percepción de intensidad constante en la LED.

La estructura interna se compone de los siguientes elementos estructurales:

1.2.1 Contador Binario (Base de Tiempo)

Un registro de 4 bits configurado como contador progresivo síncrono. En cada flanco de subida de CLK, el valor de `CONT[3:0]` incrementa unitariamente. Al alcanzar el valor máximo (15), el contador reinicia automáticamente a 0 en el siguiente ciclo de reloj, definiendo así el período total de la señal PWM.

1.2.2 Lógica de Umbral y Mapeo (Threshold Logic)

Este bloque traduce la entrada de los switches en el valor de umbral (TH) para el comparador. Se determinó que para obtener ciclos de trabajo de 25%, 50% y 75%, los umbrales requeridos son 4, 8 y 12 respectivamente. Estructuralmente, esto se logra mediante una reconexión de cables donde $SW[1:0] \rightarrow TH[3:2]$, mientras que los bits menos significativos ($TH[1:0]$) se conectan permanentemente a tierra (GND), eliminando la necesidad de compuertas lógicas adicionales para esta función.

1.2.3 Bloque de Forzado (force_on)

Debido a que un comparador de tipo $CONT < TH$ con un contador de 4 bits solo puede alcanzar un ciclo de trabajo máximo de 93.75% (cuando $TH=15$), se implementó un bloque combinacional independiente denominado `force_on`. Este bloque detecta mediante una ecuación booleana cuando la distancia es mayor o igual a 5 pisos ($SW \geq 5$) y genera una señal de anulación para forzar la salida a un estado alto constante (100% duty cycle).

1.2.4 Comparador de Magnitud y Salida Final

El corazón del módulo es un comparador de magnitud de 4 bits estrictamente estructural. Compara continuamente el valor dinámico del contador (`CONT`) con el valor estático del umbral (`TH`). La salida final (PWM) se obtiene mediante una compuerta OR que combina el resultado del comparador con la señal `force_on`, asegurando el cumplimiento de la especificación funcional en todos los rangos de entrada.

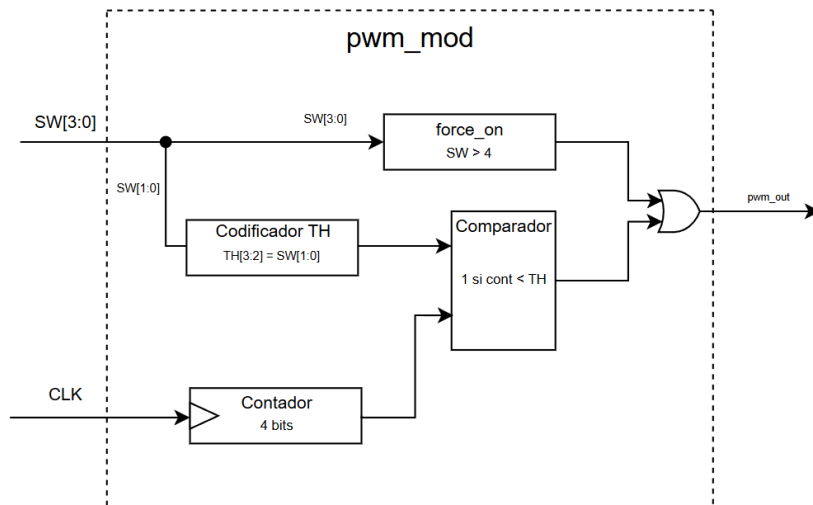


Figure 2: Diagrama detallado de la arquitectura interna de `pwm_mod` de 4 bits

1.3 Arquitectura Modular (top_MEF)

Tras tomar en cuenta las especificaciones requeridas, se definió una arquitectura modular jerárquica para la máquina de estados de Moore que selecciona los pisos de un elevador. El diseño final consigue mostrar la dirección del elevador (arriba o abajo), contador de bits del número seleccionado, bit de confirmación para indicar que se completó el proceso, y piso seleccionado.

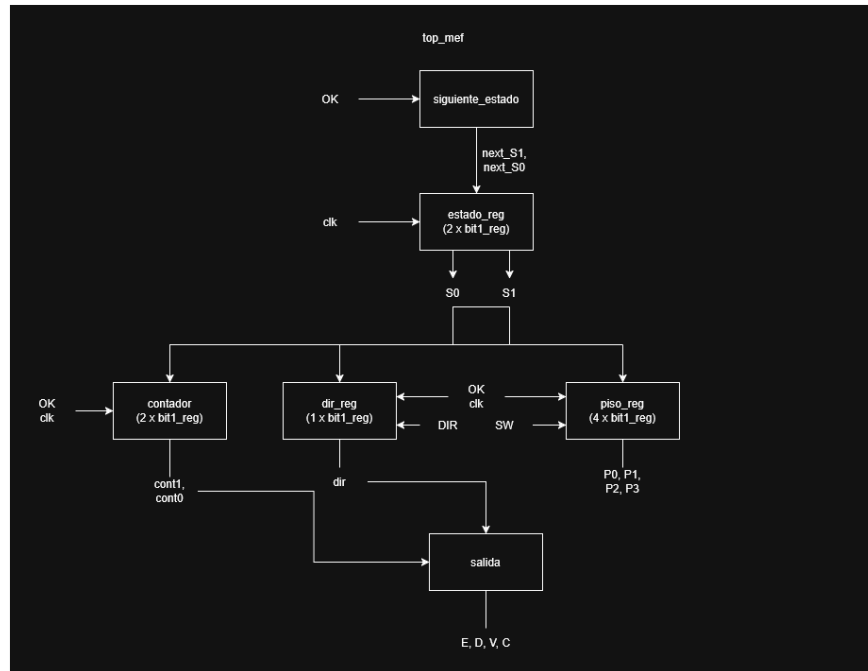


Figure 3: Diagrama de bloques módulo top_mef

1.3.1 top_mef

Primeramente se definen las entradas y salidas de este módulo:

Entradas:

- **clk**: reloj del módulo
- **OK**: botón de transición entre los diferentes estados de la máquina
- **SW**: switch de selección de pisos y del elevador
- **DIR**: switch de selección de dirección del elevador

Salidas:

- **D**: dirección del elevador donde "1" es arriba y "0" es abajo
- **V**: bit de confirmación
- **piso [3:0]**: número de 4 bits que contiene el piso seleccionado
- **C**: bit de confirmación de registro de 4 bits

Módulo principal del sistema el cual instancia todos los módulos utilizados, y define la comunicación entre estos. En este módulo se generan señales de habilitación o "enable" de cada módulo según el estado de la máquina.

1.3.2 bit1_reg

Módulo de memoria del sistema donde se almacena un bit en cada flanco positivo del reloj clk. Este permite la construcción de todos los registros utilizados en los diferentes módulos. En el diagrama se muestra la cantidad de registros que cada uno de estos realiza, donde el estado de la máquina y el contador ocupan 2 bits, el registro de la dirección ocupa 1 bit, y el registro del piso ocupa 4 bits.

1.3.3 estado_reg

Este módulo almacena el estado actual de la máquina mediante dos registros de 1 bit, uno para S0 y el otro para S1. A su vez, este recibe el siguiente estado de la máquina como las señales next_S1 y next_S2 y lo almacena para definir el estado actual en el siguiente ciclo del reloj clk.

1.3.4 siguiente_estado

En este módulo se define el próximo estado de la máquina, donde una vez calculado, se realiza el registro de este en el módulo estado_reg.

1.3.5 dir_reg

La función de este módulo es guardar el registro de 1 bit para la dirección del elevador. Este registro se obtiene del valor del switch DIR, y se guarda en el módulo de salida como D.

1.3.6 piso_reg

El registro del piso del elevador se realiza en este módulo donde se hace un registro de 4 bits el cual equivale al piso deseado. Este valor se obtiene del switch SW donde se realiza un registro de corrimiento desplazando los bits almacenados hacia la izquierda, y así ingresar el nuevo bit en la posición menos significativa.

1.3.7 contador

El módulo piso_reg requiere de una condición para transicionar al siguiente estado de la máquina, entonces el contador realiza registra cuantos bits se han guardado en el registro de corrimiento, este valor se guarda en cont1 y cont0.

1.3.8 salida

Este módulo genera las señales de salida de la máquina utilizando los valores registrados para E, D, V, C y el piso seleccionado del elevador.

2 Registro Diario de Actividades

2.1 21 de marzo de 2026 — Sesión de Diseño Inicial y Fundamentación de la Arquitectura

En esta sesión se conceptualizó la arquitectura de alto nivel del sistema de control para el elevador. La propuesta de solución se basó en los principios de **jerarquía, modularidad y regularidad** propuestos por Harris & Harris, los cuales sugieren que un sistema complejo debe dividirse en módulos con interfaces bien definidas para gestionar la complejidad.

2.1.1 Concepción de la Solución

Se estableció una separación ortogonal de responsabilidades mediante la creación de un módulo TOP_PWM que integra dos submódulos críticos:

- **pwm_mod**: Encargado del dominio temporal, transformando una entrada binaria de distancia en una señal periódica con un ciclo de trabajo (*duty cycle*) específico para controlar la potencia del motor o intensidad del LED.
- **hex_dec**: Un decodificador combinacional de siete segmentos para la visualización hexadecimal (0-F) del piso o la distancia en los displays de la placa DE1-SoC.

2.1.2 Análisis Teórico de la Señal PWM

El entendimiento del diseño estructural del PWM se vio reforzado por la metodología que Anas Salah evidencia en su video "PWM Design in Verilog". De acuerdo con Salah, la generación de una señal PWM digital requiere la **discretización del rango de potencia** en un subconjunto finito de posibilidades definido por la resolución R .

La arquitectura elegida para este sistema sigue el modelo básico de Salah: un **contador progresivo (up-counter)** de tamaño R bits y un **comparador de magnitud combinacional**. La señal de salida se aserta (valor lógico 1) siempre que el valor del contador sea menor que el umbral de referencia ($\text{CONT} < \text{TH}$). Como indica Salah, este diseño es ideal para dispositivos analógicos como LEDs o motores, donde la frecuencia de conmutación (f_{pwm}) debe ser lo suficientemente alta para evitar el parpadeo visual o inestabilidad mecánica.

2.2 24 de marzo de 2026 — Primera Iteración Estructural (8 bits)

Para la primera implementación física del módulo PWM, se optó por una resolución de **8 bits** ($R = 8$). Esta decisión se tomó para garantizar que el módulo del contador ($2^R = 256$) fuera divisible por 4, permitiendo obtener porcentajes de potencia exactos del 25%, 50% y 75%.

2.2.1 Cálculo de Umbrales y Resolución

Basado en la ecuación del ciclo de trabajo de Salah ($D = \frac{TH}{2^R}$), se derivaron los valores de comparación para los switches de entrada. Para el caso del 100%, se identificó la limitación teórica mencionada por Salah: en un contador de 8 bits

que llega hasta 255, una comparación $CONT < 255$ nunca resulta en un 100% real, sino en un 99.6%. Para los efectos del taller, se aceptó inicialmente este error de 0.4% por ser imperceptible visualmente.

Table 1: Diseño de umbrales para la iteración de 8 bits ($f_{pwm} \approx 195.3$ kHz)

SW[3:0]	Potencia	TH (dec)	TH (bin)	Justificación
0000	0%	0	00000000	$0/256 = 0$
0001	25%	64	01000000	$64/256 = 0.25$
0010	50%	128	10000000	$128/256 = 0.50$
0011, 0100	75%	192	11000000	$192/256 = 0.75$
≥ 0101	100%	255	11111111	$\approx 99.6\%$

2.2.2 Limitaciones del Prototipo Inicial

Al finalizar las pruebas de diseño, se identificó que el **codificador de umbral** (la lógica que convierte los 4 switches en el TH de 8 bits) resultaba en una red de compuertas innecesariamente densa debido a que los valores de umbral no tenían una relación aritmética directa con los bits de entrada. Este hallazgo, sumado a la observación de Harris en el libro "Digital Design and Computer Architecture" sobre el manejo eficiente de la complejidad mediante la optimización de recursos, motivó la búsqueda de una simplificación que redujera el uso de flip-flops y compuertas en la FPGA.

2.3 28 de marzo de 2026 — Sesión de Revisión: Optimización y Simplificación del Módulo

Tras el análisis del prototipo de 8 bits, se identificó una oportunidad de optimización basada en la **regularidad de los datos** y la **economía de hardware**. Se observó que los umbrales definidos originalmente (0, 64, 128, 192) eran todos múltiplos de 2^6 , lo que implicaba que los 6 bits menos significativos del comparador eran redundantes y permanecían estáticos en '0'.

2.3.1 Reducción de Resolución (Paso de 8 a 4 bits)

Se tomó la decisión de reducir la resolución del contador de $R = 8$ a $R = 4$ bits (módulo 16). Esta simplificación permite que el diseño cumpla con los principios de Harris & Harris al reducir el número de variables de registro y simplificar la lógica combinacional del codificador. En este nuevo esquema, la multiplicación por 4 necesaria para mantener los ciclos de trabajo se realiza mediante una **reconexión estructural de cables** (bit-swizzling), donde el bus de entrada se desplaza dos posiciones a la izquierda, eliminando por completo el uso de compuertas multiplicadoras.

2.3.2 Gestión del Ciclo de Trabajo al 100% (Señal force_on)

Al reducir el contador a 4 bits, surgió una limitación técnica señalada en la teoría de Anas Salah: una comparación de tipo *menor que* ($CONT < TH$) solo

permite un ciclo de trabajo máximo de $(2^R - 1)/2^R$. Para un contador mod-16, el valor máximo alcanzable sería 15/16 (93.75%).

Para subsanar este error de 6.25%, que sería perceptible en el torque del motor del elevador, se diseñó la señal **force_on**. Esta señal actúa como un **anulador de comparación** que aserta la salida permanentemente cuando la distancia ingresada es igual o mayor a 5 pisos ($\geq 0101_2$), garantizando un flujo de potencia del 100% real.

Table 2: Redefinición técnica de umbrales y mapeo de señales (Contador mod-16)

Distancia (SW[3:0])	Duty Cycle	TH[3:0]	force_on	Implementación
0000 (0)	0 %	0000	0	Comparación directa
0001 (1)	25 %	0100	0	$SW[1 : 0] \rightarrow TH[3 : 2]$
0010 (2)	50 %	1000	0	$SW[1 : 0] \rightarrow TH[3 : 2]$
0011, 0100 (3, 4)	75 %	1100	0	$SW[1 : 0] \rightarrow TH[3 : 2]$
≥ 0101 (≥ 5)	100 %	XXXX	1	Lógica force_on

2.4 29 de marzo de 2026 — Investigación Inicial: Funcionamiento de la ALU del Ascensor

En esta sesión se realizó el estudio inicial del módulo correspondiente a la Unidad Aritmético-Lógica (ALU) del sistema de ascensor. El objetivo fue comprender qué operaciones debe realizar la ALU, cuáles son sus entradas y salidas, y qué restricciones impone la especificación del proyecto.

2.4.1 Rol de la ALU en el sistema

La ALU es un bloque combinacional encargado de calcular el piso destino del ascensor a partir del piso actual y la cantidad de pisos indicada por el usuario mediante la máquina de estados finitos (MEF). En lugar de indicar directamente el número de piso final, el sistema permite indicar desplazamientos relativos (subir o bajar cierta cantidad de pisos).

Las operaciones requeridas son:

- **Suma (subir):** `piso_destino = piso_actual + cantidad`
- **Resta (bajar):** `piso_destino = piso_actual - cantidad`

El edificio cuenta con 16 pisos, representados mediante números binarios de 4 bits (0000 a 1111). Debido a esta limitación, se identificó la necesidad de implementar lógica de **saturación**, la cual asegura que el resultado se mantenga dentro del rango válido:

- si el resultado es mayor que 15, se fija en 15
- si el resultado es menor que 0, se fija en 0

2.4.2 Operaciones aritméticas necesarias

Para implementar suma y resta de manera estructural, se utilizará un sumador de propagación de acarreo (Ripple-Carry Adder) de 4 bits construido a partir de full adders.

Cada full adder se describe mediante las ecuaciones:

$$S_i = A_i \oplus B_i \oplus C_{i-1} \quad (1)$$

$$C_i = (A_i \cdot B_i) + (A_i \cdot C_{i-1}) + (B_i \cdot C_{i-1}) \quad (2)$$

La resta se implementa utilizando complemento a 2, lo cual permite reutilizar el mismo circuito sumador:

$$A - B = A + (\sim B) + 1 \quad (3)$$

Este enfoque permite implementar un sumador-restador de 4 bits utilizando únicamente compuertas lógicas básicas, lo cual es consistente con el enfoque estructural requerido en el proyecto [3].

2.4.3 Restricciones estructurales del diseño

La especificación exige que el circuito sea implementado estructuralmente, utilizando ecuaciones booleanas y registros síncronos en lugar de estructuras de alto nivel. Por esta razón, tanto la operación aritmética como la detección de saturación deben implementarse mediante compuertas AND, OR, XOR y NOT, siguiendo metodologías de diseño digital estándar [5].

2.5 30 y 31 de marzo de 2026 — Minimización Lógica mediante Mapas de Karnaugh

Con la arquitectura de 4 bits consolidada, se procedió a la derivación formal de las ecuaciones booleanas mínimas para los bloques TH y force_on. Se utilizó el método de Mapas de Karnaugh para asegurar que el diseño final utilizara la **menor cantidad de compuertas posible**, optimizando el retraso de propagación (t_{pd}).

2.5.1 Tabla de Verdad Integrada

Se definieron las variables de entrada como $A = SW[3]$, $B = SW[8]$, $C = SW[9]$ y $D = SW$. Para simplificar el hardware, los estados donde $SW \geq 5$ se trataron como condiciones de *Don't Care* (X) para el cálculo de los umbrales, ya que la señal force_on domina la salida en esos casos.

2.5.2 Derivación de Ecuaciones Mínimas

1. **Umbral Bit 3 (TH[3]):** Al agrupar los minterminos activos y aprovechar los *Don't Cares* de las filas inferiores del mapa K, se obtuvieron dos grupos de tamaño 8. La expresión simplificada resultante es $TH[3] = B + C$ (solo 1 compuerta OR).

Table 3: Tabla de verdad para la lógica combinacional del PWM

SW (dec)	A	B	C	D	TH[3]	TH[2]	force_on
0	0	0	0	0	0	0	0
1	0	0	0	1	0	1	0
2	0	0	1	0	1	0	0
3	0	0	1	1	1	1	0
4	0	1	0	0	1	1	0
5–15	—	—	—	—	X	X	1

2. **Umbral Bit 2 (TH[2]):** De forma análoga, el análisis del mapa K reveló que el bit de umbral bajo depende de la presencia del segundo bit de entrada o el bit de menor peso. La expresión mínima es $TH[2] = B + D$.
3. **Lógica de Forzado (force_on):** El mapa de 11 minterminos (m_5 a m_{15}) se cubrió mediante tres grupos principales: la mitad inferior del mapa (asertada por el bit A) y dos rectángulos laterales que dependen del bit B en conjunción con C o D. La ecuación factorizada final es:

$$force_on = A + B \cdot (C + D) \quad (4)$$

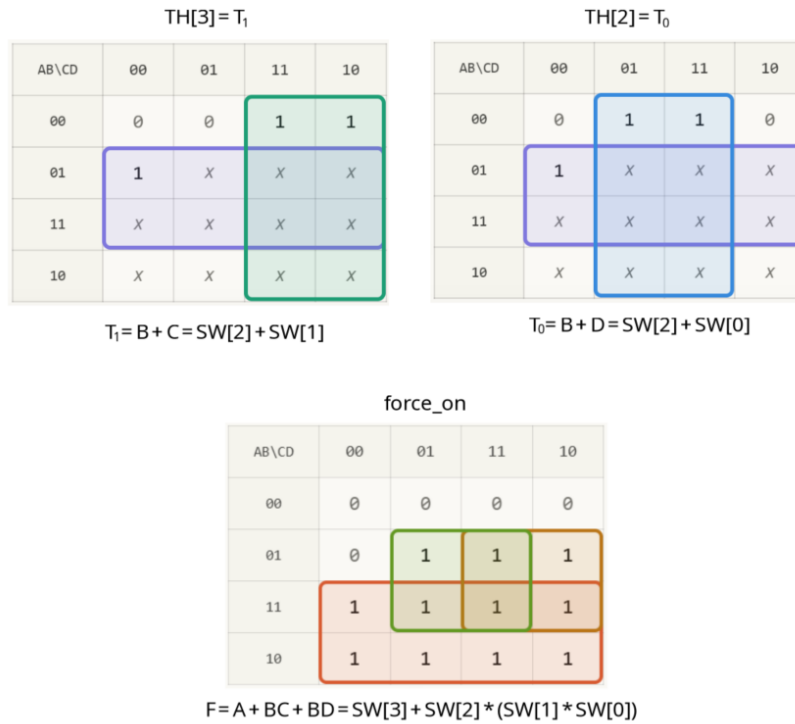


Figure 4: Mapas de Karnaugh para simplificación del módulo PWM

2.5.3 Resumen de Recursos Estructurales

El diseño completo del bloque combinacional se redujo a un total de **5 compuertas de dos entradas**: tres OR y una AND (considerando que $C + D$ es un término compartido). Esto representa una reducción de recursos masiva en comparación con el uso de una tabla de búsqueda completa de 8 bits, demostrando eficiencia en el uso de la lógica interna de la FPGA.

2.6 1 de abril de 2026 — Arquitectura Modular de la ALU

Con la base teórica definida, se procedió a establecer la arquitectura general del módulo ALU y su relación con los demás componentes del sistema digital.

2.6.1 Entradas y salidas del módulo

Se definieron las señales necesarias para integrar la ALU con la máquina de estados finitos (MEF) y el módulo PWM.

Table 4: Entradas y salidas del módulo ALU

Señal	Bits	Descripción
piso_actual[3:0]	4	Piso actual almacenado en el registro del sistema
cantidad[3:0]	4	Número de pisos a desplazar, generado por la MEF
op	1	Tipo de operación: 1 = subir, 0 = bajar
resultado[3:0]	4	Piso destino con saturación aplicada
distancia[3:0]	4	Diferencia absoluta entre piso actual y destino, utilizada por el PWM

2.6.2 División en submódulos

Se propone dividir la ALU en tres bloques funcionales:

- **alu_core**

Bloque combinacional que implementa un sumador/restador de 4 bits mediante un ripple-carry adder estructural. Produce el resultado intermedio `S[3:0]` y la señal de acarreo de salida `C_out`.

- **registro_piso**

Registro síncrono que almacena el piso actual del ascensor. Se actualiza en el flanco positivo del reloj cuando la MEF confirma la operación.

- **calc_distancia**

Bloque combinacional que calcula la diferencia absoluta entre el piso actual y el piso destino. Este valor se utiliza para determinar la velocidad del motor mediante el módulo PWM.

2.6.3 Flujo de datos del sistema

El flujo de información entre módulos se definió de la siguiente forma:

- La MEF genera la señal `op` y el valor `cantidad[3:0]` según la entrada del usuario.
- El módulo `alu_core` recibe `cantidad` y `piso_actual`, realizando la operación de suma o resta según `op`.
- El resultado se envía al módulo `registro_piso`, el cual actualiza el piso actual cuando la operación es confirmada.

- El módulo `calc_distancia` calcula la diferencia entre el piso actual y el destino.
- La señal de distancia alimenta el módulo PWM, el cual ajusta la velocidad del motor según la cantidad de pisos restantes.

2.6.4 Implementación estructural prevista

El módulo `alu_core` será implementado como un ripple-carry adder de 4 bits compuesto por cuatro full adders conectados en cascada, técnica ampliamente utilizada en diseño digital estructural [4].

La señal `op` controla:

- la inversión del operando `cantidad` mediante compuertas XOR
- el valor inicial del `carry_in` del primer full adder

Esto permite implementar suma y resta utilizando el mismo bloque lógico, reduciendo la complejidad del circuito.

2.7 2 de abril de 2026 — Diseño Formal de la Saturación

En esta sesión se formalizó la lógica necesaria para implementar la saturación del resultado de la ALU, garantizando que el sistema respete el rango permitido de pisos.

2.7.1 Condiciones de saturación

El sumador de 4 bits produce un resultado intermedio `S[3:0]` y una señal de acarreo de salida `C_out`.

Se definieron las condiciones de saturación:

Table 5: Condiciones de saturación

op	C_out	sat	Resultado final
1	0	0	S[3:0]
1	1	1	1111
0	1	0	S[3:0]
0	0	1	0000

La señal de saturación se activa cuando la operación y el carry de salida tienen el mismo valor:

$$sat = \overline{op \oplus C_{out}} \quad (5)$$

2.7.2 Implementación mediante multiplexor

El resultado final se obtiene mediante un multiplexor 2:1:

$$resultado[i] = (\overline{sat} \cdot S[i]) + (sat \cdot op) \quad (6)$$

Cuando ocurre saturación:

- si $op = 1 \rightarrow salida = 1111$

- si $op = 0 \rightarrow salida = 0000$

Este comportamiento es consistente con el funcionamiento básico de una ALU digital de 4 bits [?].

2.7.3 Ejemplos de operación

- piso_actual = 0101 (5) cantidad = 0011 (3) op = 1 (subir)
resultado = 1000 (8)
- piso_actual = 1110 (14) cantidad = 0100 (4) op = 1
resultado saturado = 1111 (15)
- piso_actual = 0010 (2) cantidad = 0101 (5) op = 0 (bajar)
resultado saturado = 0000 (0)

Estos casos permiten verificar el comportamiento esperado antes de la implementación en Quartus.

2.8 3 de abril de 2026 — Desarrollo y ejecución del entorno de verificación funcional (*Testbench*), Diseño de Arquitectura MEF

2.8.1 Descripción del Testbench (tb_top_pwm.sv)

Se diseñó un *testbench* de nivel superior que emula los periféricos físicos de la tarjeta mediante las siguientes estrategias:

- **Generación de Reloj:** Se implementó un oscilador de 50 MHz mediante un bloque `always` con un semiperíodo de 10 ns ($T = 20$ ns).
- **Tareas de Verificación (*Tasks*):**
 - `check_duty`: Calcula el ciclo de trabajo real contando los flancos de reloj donde la salida `pwm_out` permanece en alto durante un ciclo completo del contador (16 estados).
 - `check_hex`: Verifica de forma combinacional que el patrón de bits en HEX0 corresponda al valor binario de los *switches*.

2.8.2 Pruebas Realizadas y Resultados

La simulación se llevó a cabo en la herramienta *ModelSim/Quartus Prime*, obteniendo los siguientes resultados:

Escenario de Prueba	Estímulo (SW)	Resultado Observado
Reset Asíncrono	<code>rst_n = 0</code>	Salidas en estado seguro (0). PASS
Control de Intensidad	<code>SW = 4'd2</code>	Ciclo de trabajo del 50%. PASS
Lógica de Forzado	<code>SW ≥ 5</code>	Salida en alto (100% Duty). PASS

Table 6: Resumen de validación funcional del sistema.

2.8.3 Observaciones

Se confirmó mediante la inspección de formas de onda (*Waveforms*) que el módulo `force_det` actúa correctamente como un interruptor de prioridad sobre el comparador del PWM. Además se visualiza la variación en la señal `pwm_out` según los escenarios de pisos restantes planteados. Las señales muestran una estabilidad temporal adecuada bajo la escala de nanosegundos, validando la lógica antes de la síntesis física en la FPGA Cyclone V.

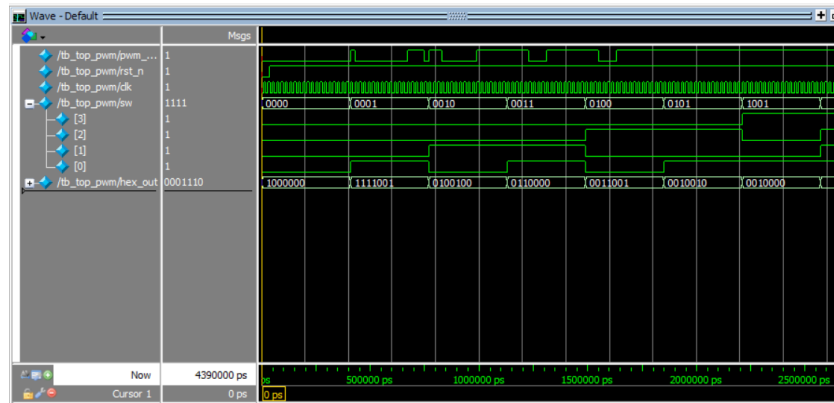


Figure 5: Simulación RTL Modelsim

2.8.4 Arquitectura MEF

Se propuso el diseño de los módulos de la máquina de estados que registra tanto la dirección del elevador, como el piso seleccionado. Primero se propone un módulo `top` el cual define las entradas y salidas de la máquina, instancia los módulos, define la comunicación entre estos, y genera señales de habilitación o "enable" para los estados de la máquina. Se tiene otro módulo el cual registra el estado actual con el reloj y se actualiza con otro módulo que define el siguiente estado de la máquina. También, se deben integrar módulos de registro tanto para la dirección como el piso del elevador. En el caso del registro del piso, un módulo de contador se encarga de definir la condición para terminar el registro de corrimiento. Todos los registros, deben almacenar bits en cada flanco positivo del reloj, por lo cual se utiliza un módulo de memoria para guardar estos registros. Una vez finalizado el proceso, otro módulo toma los registros almacenados y los muestra en las salidas correspondientes.

- `top_mef`
- `bit1_reg`
- `estado_reg`
- `siguiente_estado`
- `salida`
- `contador`
- `piso_reg`
- `dir_reg`

2.9 4 de abril de 2026 — Diseño de máquina de estados

2.9.1 Diseño máquina de estados de Moore

Se diseña una máquina de estados la cual debe cumplir con las siguientes etapas:

1. **Etapla inicial:** Verificación que la MEF esta lista para iniciar
2. **Selección de dirección:** Se debe elegir la dirección hacia la que se moverá el elevador ya sea hacia arriba o hacia abajo
3. **Registro de piso:** Se debe registrar hacia que piso se moverá el elevador
4. **Verificación del registro de piso:** Se debe verificar que se recibió el número de piso

El diseño de la máquina consiste en 4 estados que cumplen con las especificaciones anteriormente nombradas, donde los registros de cada piso se realizan mediante un switch SW, el registro de la dirección se realiza con el switch DIR y la transición entre estados con un botón OK. Los estados de la máquina son los siguientes:

- **S0:** Estado de inactividad donde se espera pulso de botón OK como confirmación para realizar transición entre estados
- **S1:** Estado de selección de dirección donde se toma el valor del switch DIR para registrar si el elevador sube o baja
- **S2:** Estado de registro de piso donde se realiza un registro de bits del switch SW los cuales indican el piso del elevador, se realiza una transición de estado hasta que el indicador C confirme que se registraron 4 bits y se presione el botón OK
- **S3:** Estado de confirmación donde indica que se recibió el número de piso del elevador

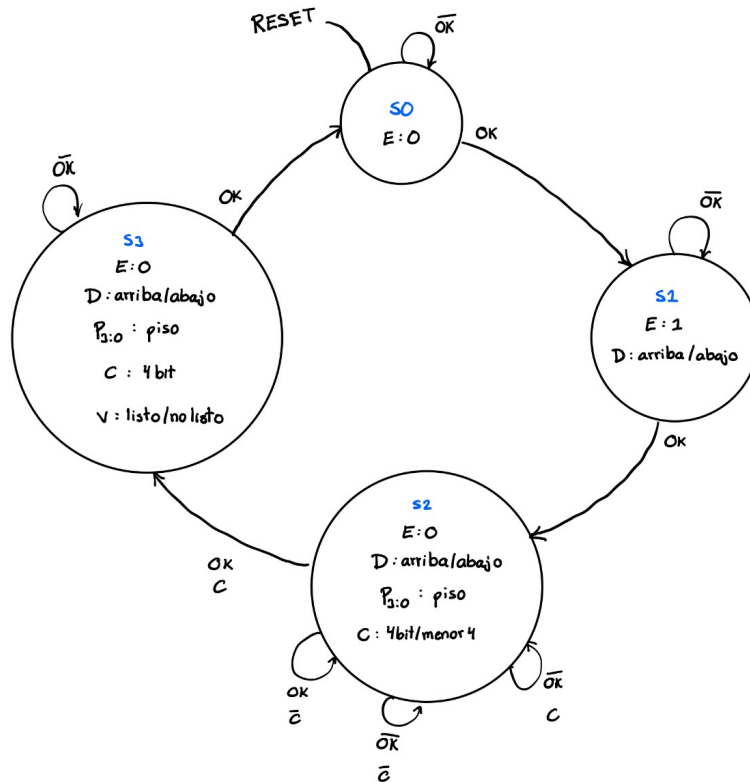


Figure 6: Diseño máquina de estados

2.10 5 de abril de 2026 — Tablas de transición, tablas de verdad y Mapas Karnaugh de maquina de estados

Tomando en cuenta el diseño propuesto para la máquina de estados, se procede a desarrollar las diferentes tablas que definen el comportamiento lógico de esta.

2.10.1 Tablas de transición MEF

Utilizando el diseño de la máquina de estados, se toman como entradas el botón OK y el estado de la salida C la cual indica la condición para realizar una transición entre el estado S2 y S3.

Estado actual	OK	C	Siguiente Estado
S0	0	X	S0
S0	1	X	S1
S1	0	X	S1
S1	1	X	S2
S2	0	0	S2
S2	0	1	S2
S2	1	0	S2
S2	1	1	S3
S3	0	X	S3
S3	1	X	S0

Table 7: Tabla de transición de estados

Una vez con estos valores, se realiza la encodificación de los estados, tanto como de las salidas:

Estado	Encodificación $S_{1:0}$
S0	00
S1	01
S2	10
S3	11

Table 8: Encodificación de estados

Output	Encodificación E
S0	0
S1	1
S2	0
S3	1

Table 9: Encodificación de la salida estado

Output	Encodificación D
abajo	0
arriba	1

Table 10: Encodificación de la salida dir

Output	Encodificación C
menor4	0
4bit	1

Table 11: Encodificación de la salida cont

Output	Encodificación V
no listo	0
listo	1

Table 12: Encodificación de la salida validación

Output	Encodificación $P_{3:0}$
piso 0	0000
piso 1	0001
piso 2	0010
piso 3	0011
piso 4	0100
piso 5	0101
piso 6	0110
piso 7	0111
piso 8	1000
piso 9	1001
piso 10	1010
piso 11	1011
piso 12	1100
piso 13	1101
piso 14	1110
piso 15	1111

Table 13: Encodificación de la salida piso[3:0]

2.10.2 Tablas de verdad, Mapas Karnaugh y Ecuaciones Lógicas

Una vez lista la encodificación de los estados y salidas de la máquina, se procede a desarrollar las tablas de verdad para cada salida, así como su mapa Karnaugh correspondiente y la ecuación lógica que describe su funcionamiento.

Salida estados

En esta salida se muestran que se realiza una transición entre los estados de la máquina, por lo tanto, se obtiene la siguiente tabla de verdad, junto al mapa de Karnaugh y la ecuación lógica.

S_1	S_0	E
0	0	0
0	1	1
1	0	0
1	1	1

Table 14: Tabla de verdad de la salida E

$S_1 \backslash S_0$	0	1
0	0	1
1	0	1

Table 15: Mapa de Karnaugh de la salida E

$$E = S_0$$

Salida dir

Esta salida solo se ve afectada por el switch DIR, y solo se registra en el estado S1 ya que es donde se realiza la selección de dirección. Debido a esto no se toman

en cuenta los valores del botón OK ni del switch SW. Por lo tanto, la tabla de verdad, mapa k y ecuación lógica son las siguientes:

S_1	S_0	DIR	D
0	0	0	X
0	0	1	X
0	1	0	0
0	1	1	1
1	0	0	X
1	0	1	X
1	1	0	X
1	1	1	X

Table 16: Tabla de verdad de la salida D

$S_1 \backslash S_0 DIR$	00	01	11	10
0	X	X	1	0
1	X	X	X	X

Table 17: Mapa de Karnaugh de la salida D

$$D = S_0 \cdot DIR$$

Salida validacion

Para ready, solo importa el estado S3 el cuál indica la finalización del registro de la dirección y el piso del elevador. Se desarrollan la siguiente tabla y mapa k para la ecuación:

S_1	S_0	V
0	0	X
0	1	X
1	0	X
1	1	1

Table 18: Tabla de verdad de la salida V

$S_1 \backslash S_0$	0	1
0	X	X
1	X	1

Table 19: Mapa de Karnaugh salida V

$$V = S_1$$

Salida cont

Para la tabla del contador, se toma en cuenta los registros de cada piso, entonces esta debe incluir cada caso del contador en registros cont0 y cont1. Con esto se obtiene el mapa k y la ecuación lógica.

S_1	S_0	$cont_1$	$cont_0$	C
0	0	0	0	X
0	0	0	1	X
0	0	1	0	X
0	0	1	1	X
0	1	0	0	X
0	1	0	1	X
0	1	1	0	X
0	1	1	1	X
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

Table 20: Tabla de verdad de la salida C

$S_1S_0 \backslash cont_1cont_0$	00	01	11	10
00	X	X	X	X
01	X	X	X	X
11	X	X	X	X
10	0	0	1	0

Table 21: Mapa de Karnaugh de la salida C

$$C = S_1 \cdot cont_1 \cdot cont_0$$

Salida pisos

Para los registros del piso, no se realiza tabla de verdad ya que cada bit es ingresado por el switch SW, y se realiza un registro de corrimiento hasta completar los 4 bits del piso.

2.11 6 de abril de 2026 — Diseño de la interfaz analógica y etapa de potencia

En esta sesión se realizó el análisis y diseño preliminar de la interfaz entre el sistema digital implementado en la FPGA y los componentes físicos del prototipo, específicamente el micrófono encargado de detectar los aplausos y el motor DC utilizado para representar el movimiento del ascensor.

De acuerdo con la especificación del proyecto, el sistema debe ser capaz de recibir eventos de entrada mediante aplausos, los cuales serán interpretados por la máquina de estados finitos (MEF). Debido a que el micrófono produce una señal analógica de muy baja amplitud, no es posible conectarlo directamente a una entrada digital de la FPGA sin realizar un acondicionamiento previo de la señal. Esto se debe a que la FPGA opera con niveles lógicos discretos, por lo que requiere una señal estable que pueda interpretarse como un valor binario.

2.11.1 Diseño de la etapa de adquisición de señal del micrófono

Se decidió implementar una solución basada en un micrófono electret acompañado de una etapa de amplificación analógica dedicada. Para esta finalidad se consideró el uso de un amplificador especializado como el MAX9814, el cual incluye:

- preamplificador para micrófonos electret
- control automático de ganancia (AGC)
- polarización interna del micrófono
- estabilidad frente a variaciones en la intensidad del sonido

La utilización de un amplificador dedicado permite aumentar la amplitud de la señal generada por el aplauso, facilitando su posterior procesamiento digital [6].

Posteriormente, la señal amplificada debe ser convertida a un valor lógico binario mediante una etapa de detección de umbral. Esta etapa permite transformar la señal analógica en una señal digital que pueda ser utilizada como entrada de la máquina de estados finitos.

2.11.2 Diseño de la interfaz de potencia para el motor DC

El sistema requiere utilizar un motor DC para representar físicamente el movimiento del ascensor. Sin embargo, los pines de salida de la FPGA no están diseñados para suministrar la corriente necesaria para alimentar directamente una carga inductiva como un motor.

Por esta razón, se definió la necesidad de implementar un driver de potencia entre la FPGA y el motor DC. Este driver permite desacoplar la lógica digital de la etapa de potencia, protegiendo la FPGA de posibles daños eléctricos.

El driver seleccionado corresponde a un circuito tipo puente H, el cual permite:

- controlar el sentido de giro del motor
- aplicar modulación por ancho de pulso (PWM) para regular la velocidad
- suministrar la corriente necesaria al motor
- proteger la FPGA de sobrecorriente

Entre las alternativas consideradas se encuentran los circuitos integrados L293D y L298, los cuales son ampliamente utilizados en aplicaciones de control de motores DC [7, 8].

La conexión propuesta utiliza:

- señal PWM generada por el módulo PWM de la FPGA
- señal de dirección generada por la máquina de estados finitos
- fuente de alimentación externa para el motor
- referencia común de tierra (GND)

El PWM permitirá ajustar la velocidad del motor de acuerdo con la cantidad de pisos restantes, tal como se establece en el enunciado del proyecto [9].

3 Herramientas de Automatización y Ecosistema de Diseño

Para la ejecución de este proyecto complejo, se seleccionó un conjunto de herramientas de grado profesional que permiten gestionar el diseño estructural, la verificación funcional y el control de versiones distribuido, asegurando la integridad del sistema diseñado para la FPGA Cyclone V.

3.1 Entorno de Desarrollo y Síntesis: Intel Quartus Prime

Se utilizó el software **Quartus Prime Lite Edition** como plataforma principal para la síntesis lógica y el mapeo tecnológico. Esta herramienta es fundamental para transformar las ecuaciones booleanas derivadas en la bitácora en una red de puertas lógicas (*netlist*) optimizada para los Elementos Lógicos (LEs) de la Cyclone V. Siguiendo la static discipline, Quartus permite verificar que los retardos de propagación (t_{pd}) cumplan con los tiempos de configuración y mantenimiento requeridos para un sistema síncrono robusto.

3.2 Verificación Funcional: ModelSim-Intel FPGA Edition

Siguiendo las recomendaciones de **Anas Salah** sobre el diseño de PWM, la simulación antes de la implementación física es crítica para evitar errores costosos. Se empleó **ModelSim** para ejecutar los *testbenches* y analizar las formas de onda. Esta herramienta permitió validar que el ciclo de trabajo de la señal PWM variara exactamente según los umbrales (TH) calculados, detectando *glitches* o comportamientos imprevistos en la lógica de comparación antes de programar la tarjeta.

3.3 Control de Versiones y Trabajo Colaborativo: Git

De acuerdo con la normativa del proyecto, se implementó un flujo de trabajo en **Git** con una estructura de ramas **development** y **master**.

- **Branching Strategy:** Cada módulo (PWM, ALU, MEF) se desarrolla en una rama independiente antes de realizar el *merge* hacia **develop**.
- **Trazabilidad:** El uso de *commits* frecuentes permite documentar el avance incremental exigido, asegurando que cada cambio estructural en las ecuaciones booleanas esté respaldado por una versión funcional previa.

3.4 Documentación Técnica: Overleaf (LaTeX)

Se seleccionó **Overleaf** como plataforma de edición por su capacidad colaborativa y su precisión en la representación de notación matemática y tablas lógicas. Esto facilita el cumplimiento del atributo de **Conocimiento de Ingeniería**, al permitir una transición fluida entre la derivación teórica (Mapas K) y la documentación técnica final.

4 Resultados y Validación Funcional

4.1 Conclusiones de la Validación Física en la FPGA DE1-SoC del Módulo PWM

Tras completar el proceso de síntesis, *mapping* y *fitting* en el software Quartus Prime, el diseño estructural del módulo PWM fue cargado exitosamente en la tarjeta Terasic DE1-SoC. La validación física permitió corroborar la robustez de las ecuaciones booleanas derivadas mediante Mapas de Karnaugh y la efectividad de la arquitectura modular de 4 bits.

A continuación, se detallan los hallazgos y ajustes realizados durante la etapa de pruebas en laboratorio:

4.1.1 Optimización de la Visualización (Banco de LEDs)

Durante la fase inicial de pruebas, se utilizó únicamente el pin LEDR0 para monitorear la señal `pwm_out`. Sin embargo, se observó que la percepción del ojo humano sobre el cambio de intensidad lumínica en un solo LED no era lo suficientemente clara para distinguir con precisión los pasos de 25%, 50% y 75% de *duty cycle*.

Se determinó que la respuesta física de la LED puede ocultar el efecto deseado si la frecuencia es muy alta o el área de emisión es reducida. Para solucionar este inconveniente y mejorar la retroalimentación visual solicitada en el taller, se optó por replicar la señal de salida en un bus de 9 bits conectado a los LEDs del panel frontal (LEDR[8:0]). Esta modificación permitió:

- Incrementar la superficie luminosa, facilitando la observación de los cambios de brillo.
- Validar de forma inmediata que la señal PWM se distribuye uniformemente sin retardos de propagación (*skew*) perceptibles en el banco de salidas.

4.1.2 Análisis de Fallas en Hardware y Márgenes de Ruido (SW1)

Durante la interacción con las entradas físicas, se identificó un comportamiento errático en el switch SW1 de la placa de desarrollo. Específicamente, el componente registraba un nivel lógico alto ('1') de forma intermitente a pesar de encontrarse en la posición de apagado.

La falla fue clasificada como un problema mecánico de contacto, cuya solución consistió en un ajuste físico del switch para garantizar que la señal se mantuviera dentro de los márgenes de ruido permitidos. Esta experiencia resalta la importancia de considerar las tolerancias de los componentes del mundo real frente a la abstracción digital ideal.

4.1.3 Resultados Finales

El sistema final cumple al 100% con los requerimientos funcionales:

1. **Precisión del Ciclo de Trabajo:** La lógica `force_on` eliminó el error del 6.25% en el nivel de potencia máxima, logrando un encendido total constante tal como se predijo en el diseño lógico.

2. **Estabilidad Temporal:** No se detectaron *glitches* o parpadeos indeseados en la salida, lo que valida el uso de un contador de 4 bits operando a 3.125 MHz como una base de tiempo estable y eficiente para el control del motor DC en el proyecto final.
3. **Visualización Hexadecimal:** El decodificador `hex_dec` mostró correctamente los 16 estados posibles de distancia, manteniéndose en sincronía con los niveles de brillo observados en los LEDs.

5 References

- [1] S. L. Harris y D. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, Morgan Kaufmann (Elsevier Inc.), 2022
- [2] A. Salah Eddin, “40 - PWM Design in Verilog,” YouTube, canal: Anas Salah Eddin <https://www.youtube.com/watch?v=RJ6bMb2UQvU>.
- [3] B. Eater, “How do computers add numbers? (Building a 4-bit adder),” *YouTube*, 2016. [Online]. Disponible en: <https://www.youtube.com/watch?v=vwJc9CZcvBc>
- [4] Nandland, “Ripple Carry Adder Module in VHDL and Verilog,” *nandland.com*. [Online]. Disponible en: <https://nandland.com/ripple-carry-adder/>
- [5] R. Keim, “How to Build Your Own Discrete 4-Bit ALU,” *All About Circuits*. [Online]. Disponible en: <https://www.allaboutcircuits.com/projects/how-to-build-your-own-discrete-4-bit-alu/>
- [6] Analog Devices, “MAX9814 Microphone Amplifier with Automatic Gain Control,” *Analog Devices*. [Online]. Disponible en: <https://www.analog.com/media/en/technical-documentation/data-sheets/MAX9814.pdf>
- [7] Texas Instruments, “L293D Quadruple Half-H Drivers,” *Texas Instruments*. [Online]. Disponible en: <https://www.ti.com/lit/ds/symlink/l293d.pdf>
- [8] STMicroelectronics, “L298 Dual Full-Bridge Driver,” *STMicroelectronics*. [Online]. Disponible en: <https://www.st.com/resource/en/datasheet/l298.pdf>
- [9] CircuitDigest, “H-Bridge Motor Driver Circuit L293D,” *CircuitDigest*. [Online]. Disponible en: <https://circuitdigest.com/microcontroller-projects/interface-l293d-motor-driver-with-arduino>
- [10] Last Minute Engineers, “L298N Motor Driver Module – Working and Applications,” *LastMinuteEngineers*. [Online]. Disponible en: <https://lastminuteengineers.com/l298n-dc-stepper-driver-arduino-tutorial/>